Lab 3: Component Modelling & Architectural Pattern Selection

Name: Anjanaa Vijaysankar | SRN: PES1UG24CS654 | Section: 5K

Problem Statement #46: Remote Team Time-Tracking & Project Approver

---

Architecture Selection

I chose **Microservices Architecture** for the Remote Team Time-Tracking & Project Approver System.

The system is split into independently deployable services: Timesheet Manager, Jira Integration Service, Approval Workflow Service and Budget Monitoring Service, with a Web Dashboard as the client for both developers and managers. The Timesheet Manager owns the Timesheet Database, and the other services access timesheet data only through its interfaces.

Reasons for the Choice

- **1. Isolating the external Jira dependency:** Every time entry has to be linked to a valid Jira task, so the system depends on an external system it does not control. Keeping all Jira communication inside a separate Jira Integration Service means that if Jira is slow or down, only task validation is affected. Developers can still view timesheets and managers can still approve them, which a single layered or client-server application could not guarantee.
- **2. Uneven load across features:** Time logging and approvals are light, steady operations, but budget burn dashboards are read constantly by managers and spike at the end of each week when timesheets are submitted. Microservices let the Budget Monitoring Service be scaled and changed on its own (for example, new burn-rate charts) without redeploying the timesheet or approval logic.

Security Advantage

Each service only holds the data and credentials it needs (least privilege). The Jira API token is stored only in the Jira Integration Service, so a flaw in the dashboard or budget service cannot expose it. Approval actions go only through the Approval Workflow Service, which checks the manager role before changing a timesheet's status, and approved timesheets are locked by the Timesheet Manager so developers cannot edit them after approval.

Performance Benefit

NFR-001 requires approval status changes and budget burn charts to update across dashboards within 500 ms. Because the Budget Monitoring Service is independent, it can be given extra instances during peak periods and can push updates to the Web Dashboard over WebSockets as soon as the Timesheet Manager reports new hours. Heavy budget recalculations therefore never slow down time entry or approvals for other users.

Alternatives Considered

A Layered architecture would have been simpler to build, but all features would scale and fail together, and a Jira outage could block the whole business layer. A Client-Server design gives easy data consistency, but its central server is a single point of failure and a bottleneck for real-time dashboards used by remote teams. The main trade-off of microservices is higher operational complexity and network latency between services, which we accept because fault isolation and independent scaling matter more for this system.